Kern-Monitor

Machbarkeitsstudie: alle Prozessorkerne nutzen und überwachen – Projektbericht mit Erkenntnissen, Messwerten und Installationshinweisen

≈ 100×
schneller als ein Python-Prozess (Rust SIMD, 48 Threads)

11
CPU-Rechenverfahren mit bitgenau gleichem Bild (Python, C#, Rust, C, C++, Assembler)

2,1×
maximale Beschleunigung bei Pi – das Amdahlsche Gesetz in der Praxis

Stand: 9. November 2026 · Referenzrechner: AMD Ryzen Threadripper 3960X (24 Kerne / 48 Threads), 64 GB RAM, NVIDIA GeForce RTX 3090, Windows 11 · zusätzlich Linux (Debian / Ubuntu)

Inhalt

1. Ziel und Ergebnis in Kürze
2. Starten, Aufbau und Funktionen
3. Erkenntnisse: Mandelbrot, Skalierung, Platzierung und SMT
4. Sprachvergleich: Python, C#, Rust, C, C++, Assembler – und SIMD
5. Klügerer Algorithmus (Kardioide) und Grafikkarte (GPU)
6. Pi und das Amdahlsche Gesetz
7. Gefundene und behobene Fehler
8. Weitergabe: Pakete Full und Lite, Installation
9. Grenzen und Ideen
10. Linux-Fassung (Debian / Ubuntu)
11. Begriffe kurz erklärt

1. Ziel und Ergebnis in Kürze

Ziel: Ein Programm, mit dem man alle Prozessorkerne nutzen und überwachen kann, bedienbar über eine interne Website, mit einer grafischen Anzeige der Belastung jedes einzelnen Kerns. Es ist eine Machbarkeitsstudie zum Ausloten der Möglichkeiten.

Ergebnis: Der „Kern-Monitor" zeigt jeden der 48 Threads live (gruppiert nach Kernen und Cache-Gruppen), erzeugt gezielt Last und rechnet zwei echte parallele Aufgaben – die Mandelbrot-Menge und die Kreiszahl π – in vielen Varianten. Daraus ergeben sich belastbare Messungen, was Parallelisierung bringt und wo ihre Grenzen liegen.

> **Die wichtigsten Erkenntnisse**
> - **Gut teilbare Aufgaben skalieren stark:** Mandelbrot wird mit 48 Threads 18–28× schneller als mit einem.
> - **Die Programmiersprache ist fast egal:** numba (Python), C#, Rust, C, C++ und handgeschriebener Assembler rechnen pro Kern gleich schnell (± wenige Prozent). Unterschiede entstehen beim Drumherum: Python muss wegen des GIL Prozesse statt Threads verwenden (langsamer Start, Datenübergabe).
> - **SIMD bringt am meisten pro Kern:** 4 Pixel gleichzeitig (AVX2) = 3,7× schneller.
> - **Ein besserer Algorithmus schlägt mehr Kerne:** Der Kardioiden-Test macht die Gesamtansicht mit 1 Thread 6–9× schneller.
> - **Serielle Anteile begrenzen alles (Amdahl):** Bei π ist nach ≈ 2× Schluss, auch mit 48 Kernen.
> - **Die GPU** (RTX 3090) ist in *float* bis 5× schneller als die beste CPU-Variante, in *double* aber langsamer.

Referenzrechner

Komponente	Ausstattung
Prozessor	AMD Ryzen Threadripper 3960X: 24 Kerne / 48 Threads (SMT), 8 Cache-Gruppen (CCX) zu je 3 Kernen mit 16 MB L3, AVX2 + FMA, kein AVX-512, max. 3,8 GHz
Arbeitsspeicher	64 GB
Grafikkarte	NVIDIA GeForce RTX 3090, 24 GB, Direct3D 11 Compute, double-Unterstützung (1/64 der float-Leistung)
Software	Windows 11, Python 3.11.9, numba 0.68, numpy 2.4, gmpy2 2.3, .NET SDK 10, Rust 1.99, Visual Studio 2026 (MSVC 14.51)

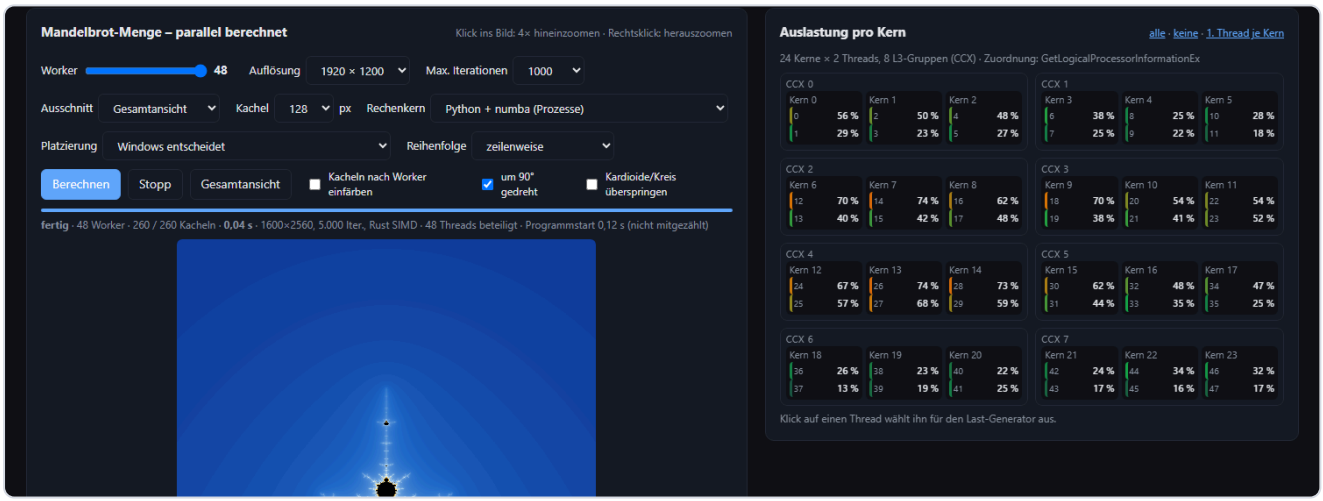
2. Starten, Aufbau und Funktionen

Starten: `start.bat` doppelklicken. Der Browser öffnet `http://127.0.0.1:8000` (nur wenn noch keine Seite offen ist; eine offene Seite lädt sich nach einem Neustart selbst neu). Der Server ist absichtlich **nur auf diesem PC erreichbar** (127.0.0.1), nicht im Netzwerk.

Sprache: Der Umschalter **DE | EN** stellt die Seite auf Deutsch oder Englisch um. Die Wahl wird gespeichert; ohne gespeicherte Wahl startet die Seite auf Englisch. Meldungen des Servers folgen der gewählten Sprache. Die Umgebungsvariable `KM_LANG` (`de` oder `en`) legt die Startsprache fest.

Funktionen der Oberfläche

- **System-Box:** beim Start erkannte Hardware (CPU, Kerne, Cache-Gruppen, Befehlssätze, RAM, Grafikkarten) und nicht verfügbare Rechenverfahren mit Grund.
- **Auslastung pro Kern:** alle Threads live (jede Sekunde), gruppiert nach Cache-Gruppe → Kern → Thread, mit 60-s-Verlauf; dazu CPU gesamt, Takt, RAM.
- **Heatmap:** alle Threads über 2 Minuten – zeigt, wie sich Last zeitlich und räumlich verteilt.
- **Mandelbrot:** 13 Rechenverfahren (siehe Abschnitt 4), 1–48 Worker, Auflösung, Iterationen, Ausschnitt, Kachelgröße, Platzierung (CPU-Affinität), Reihenfolge („aufwendige zuerst"), Kardioiden-Test, Live-Kacheln, Färbung nach Worker, Zoom per Klick, 90°-Drehung.
- **Skalierungstest:** dasselbe Bild mit 1 … 48 Workern (Median aus 1/3/5 Läufen), Diagramm als Beschleunigung oder Laufzeit, Vergleich aller Varianten; Ergebnisse als CSV (Excel) und JSON.
- **Pi:** 1 000 bis 100 Millionen Nachkommastellen, Phasenanzeige, Prüfung gegen mpmath, Download; Skalierungstest mit Phasenaufteilung.
- **Last-Generator** (beliebige Threads, Intensität, Dauer, Affinität) und **Primzahl-Benchmark**.
- **Arbeitsanzeige:** Kreisel an jeder laufenden Berechnung und echtes Lebenszeichen („keine Rückmeldung seit … s", wenn der Server nicht antwortet).
- **Sprachumschaltung DE | EN** für Oberfläche und Servermeldungen (Wahl wird gespeichert).
- Alle Erklärungen sind aufklappbar (beim Laden geschlossen).



Ausschnitt der Oberfläche: Mandelbrot-Bereich und Auslastung pro Kern (8 Cache-Gruppen × 3 Kerne × 2 Threads).

Aufbau (Dateien)

Datei / Ordner	Inhalt
`server.py`	kleine Startdatei (setzt Umgebung, startet `app.py`) – bewusst winzig, weil Worker-Prozesse sie neu laden
`app.py`	Webserver (FastAPI), Messung (psutil), WebSocket, API, Last-Generator, Primzahl-Benchmark
`hardware.py`	Hardware-Erkennung (CPU, Topologie, Befehlssätze, Grafikkarten), Skalierungsstufen, Simulation für Tests
`mandelbrot.py`	Mandelbrot: Rechenverfahren numba/numpy, Prozess-Pool, Anbindung der externen Programme, Skalierungstest
`pi.py`	Pi: Chudnovsky mit Binary Splitting (gmpy2), Phasenmessung, Prüfung, Skalierungstest
`worker_util.py`, `lastgen.py`	leichte Module für Worker-Prozesse
`static/`	Oberfläche: `index.html`, `app.js` (Kernansicht, Heatmap), `mandel.js`, `pi.js`, `style.css`
`csharp/`, `rust/`, `native/`	Vergleichsprogramme: C#, Rust, C, C++, Assembler (`asm/kernel.asm`), GPU (`gpu/mandel.hlsl`)
`benchmarks/`	Messergebnisse (CSV/JSON), Pi-Ergebnis und mpmath-Referenz
`export_*.bat`, `tools/`	Weitergabe-Pakete bauen, Installationsanleitungen

3. Erkenntnisse: Mandelbrot, Skalierung, Platzierung und SMT

Die Mandelbrot-Menge ist eine „perfekt parallele" Aufgabe: Das Bild wird in Kacheln (128 × 128 Pixel) geteilt, die unabhängig voneinander berechnet werden. Ein Prozess-Pool verteilt sie dynamisch: Wer fertig ist, holt sich die nächste Kachel. Standardmessung: 1600 × 2560 Pixel (90° gedreht), 5 000 Iterationen, Gesamtansicht, Median aus 3 Läufen.

Python + numba mit 1 … 48 Prozessen

Worker	1	2	4	8	12	16	24	32	48
Zeit (s)	6,18	3,27	1,72	0,93	0,72	0,56	0,51	0,40	0,34
Beschleunigung	1,0×	1,9×	3,6×	6,6×	8,5×	11,0×	12,1×	15,5×	18,4×

- **Python nutzt mehrere Kerne nur über Prozesse** (wegen des GIL), nicht über Threads. Der Start von 48 Prozessen dauert unter Windows 2–6 s; der Pool bleibt deshalb bestehen, und der Start wird getrennt ausgewiesen.
- **Bis 8 Worker fast linear**, danach flacht die Kurve ab. Hauptgrund ist **nicht** SMT, sondern die kurze Laufzeit: Mit 48 Workern dauert ein Bild nur noch 0,3 s – dann zählen Verwaltung und die letzten Kacheln, auf die alle warten. Mit 4-facher Arbeit (20 000 Iterationen) steigt das Ergebnis mit 48 Workern auf **24,8×**; die CPU-Zeit aller Worker bleibt dabei gleich (≈ 24 s), verloren geht reine Wartezeit.
- **Primzahlen** bis 3 Mio. (erster Prototyp): 7,0 s seriell, 0,7 s mit 48 Prozessen (10×).

Platzierung (CPU-Affinität) und SMT

Worker	2	4	8	12	16	24	48
Windows entscheidet (s)	3,27	1,72	0,93	0,72	0,56	0,51	0,34
1 Thread je Kern zuerst (s)	3,14	1,64	0,90	0,66	0,55	0,49	0,34
kompakt, beide Threads eines Kerns (s)	3,39	1,76	0,96	0,71	0,58	0,49	0,35

- **Platzierung ändert wenig:** „1 Thread je Kern zuerst“ ist bei 2–16 Workern 3–10 % schneller; bei 48 Workern sind alle gleich.
- **Windows packt wenige Worker eng zusammen:** 4 Worker landeten in Tests auf den Threads 24–27 – also auf 2 Kernen mit je beiden Threads, obwohl 22 Kerne frei waren.
- **SMT bremst Mandelbrot kaum:** 2 Worker auf einem Kern sind nur 8 % langsamer als auf zwei Kernen (bei SIMD 11 %). Grund: Jeder Rechenschritt wartet auf den vorigen; ein Thread lässt die Rechenwerke großteils ungenutzt, der zweite füllt die Lücken.

„Aufwendige Kacheln zuerst“

Kacheln im schwarzen Inneren sind rechenaufwendig (bis zur maximalen Iterationszahl). Eine grobe Aufwandsschätzung (8 × 8 Stichproben je Kachel, ≈ 15 ms, in der Zeit enthalten) sortiert sie nach vorn, damit am Ende keine lange Kachel übrig bleibt.

Worker	16	24	32	48
numba zeilenweise (s)	0,56	0,51	0,40	0,34
numba aufwendige zuerst (s)	0,54	0,40	0,36	0,29

Ergebnis: bei 24–48 Workern 10–22 % schneller (18,4× → 21,8×), bei wenigen Workern ohne Wirkung. Bei sehr kurzen Laufzeiten (SIMD, ≈ 70 ms) dauert die Schätzung länger, als sie einspart.

4. Sprachvergleich – und SIMD

Die 13 Rechenverfahren

„Rechenverfahren“ meint hier eine von 13 Varianten, dasselbe Mandelbrot-Bild zu berechnen – nicht einen Prozessorkern. Jedes Verfahren nutzt auf Wunsch alle Kerne des PCs; die GPU-Verfahren rechnen auf der Grafikkarte.

Rechenverfahren	Art	Beschreibung
Python + numba	Prozesse, 1 Pixel je Schritt	Python, von numba in Maschinencode übersetzt; wegen des GIL ein Prozess je Kern
Python + numpy	Prozesse, Feldrechnung	reines Python mit numpy, ganze Kacheln als Zahlenfelder; langsamste Variante
C# / .NET	Threads, 1 Pixel je Schritt	C# (.NET 10) mit echten Threads in einem Prozess
C# / .NET + SIMD	Threads, 4 Pixel gleichzeitig	wie C#, je Thread 4 Bildpunkte gleichzeitig (AVX2)
Rust	Threads, 1 Pixel je Schritt	Rust mit echten Threads und gemeinsamer Kachel-Warteschlange
Rust + SIMD	Threads, 4 Pixel gleichzeitig	Rust mit AVX2-Befehlen
C	Threads, 1 Pixel je Schritt	C mit Windows-Threads (CreateThread)
C + SIMD	Threads, 4 Pixel gleichzeitig	C mit AVX2-Intrinsics
C++	Threads, 1 Pixel je Schritt	C++ mit std::thread und std::atomic
C++ + SIMD	Threads, 4 Pixel gleichzeitig	C++ mit AVX2-Intrinsics
Assembler	Threads, 4 Pixel gleichzeitig	handgeschriebener x64-Assembler (AVX2) für die innere Schleife; Threads in C
GPU (double)	Grafikkarte, doppelte Genauigkeit	Direct3D-Compute-Shader, je Bildpunkt eine GPU-Recheneinheit; doppelte Genauigkeit, einzelne Pixel weichen ab (Treiber)
GPU (float)	Grafikkarte, einfache Genauigkeit	wie GPU (double), deutlich schneller, Bild weicht leicht ab

Dieselbe Aufgabe in sechs Sprachen bzw. Werkzeugen, jeweils mit echten Threads (außer Python: Prozesse) und mit SIMD (AVX2: 4 Pixel gleichzeitig). Alle Varianten liefern **bitgenau dasselbe Bild** (0 Pixel Unterschied, geprüft in mehreren Ansichten): gleiche Formel, gleiche Reihenfolge der Rechenschritte, kein FMA (zusammengezogenes Multiplizieren und Addieren würde die letzte Stelle verändern).

Threads	numba	C#	Rust	C	C++	C# SIMD	Rust SIMD	C SIMD	C++ SIMD	Assembler
1	6,18	6,09	6,21	6,08	6,06	1,65	1,63	1,66	1,67	1,69
8	0,93	0,83	0,85	0,81	0,80	0,23	0,22	0,22	0,23	0,23
24	0,51	0,40	0,40	0,39	0,39	0,11	0,10	0,11	0,11	0,11
48	0,34	0,23	0,22	0,23	0,24	0,070	**0,062**	0,070	0,071	0,072
Start	2–6 s	0,14	0,04	0,10	0,10	0,14	0,04	0,10	0,10	0,10

Zeiten in Sekunden, 1600 × 2560, 5 000 Iterationen, zeilenweise, Median aus 3. „Start" = Programm- bzw. Prozessstart (nicht mitgezählt).



Laufzeit (logarithmisch) aller Sprachen: zwei Bündel – ohne SIMD (oben) und mit SIMD (unten, gestrichelt); nur numba (blau) setzt sich ab.

- **Die Sprache ist fast egal:** Alle Compiler (MSVC, LLVM bei Rust und numba, RyuJIT bei C#) erzeugen für die innere Schleife praktisch dieselben Maschinenbefehle. Die Grenze setzt der Prozessor: Jeder Schritt wartet auf das Ergebnis des vorigen.
- **Handgeschriebener Assembler** (`native/asm/kernel.asm`, ausführlich kommentiert) ist nicht schneller als der Compiler.
- **Echte Threads statt Prozesse** bringen bei 48 Threads 23–28× statt 18× und einen Start in 0,04–0,14 s statt Sekunden.
- **SIMD** macht einen Thread **3,7× schneller** – der größte Einzelgewinn. Bestwert: **Rust SIMD mit 48 Threads, 0,062 s** ≈ 100× schneller als ein Python-Prozess. AVX-512 hat dieser Prozessor nicht.
- Mehr ginge nur mit Tricks, die in jeder Sprache gleich möglich sind: mehrere Pixelgruppen verschränkt rechnen, FMA (verändert das Bild in der letzten Stelle) oder float statt double (ungenau beim Zoomen).

5. Klügerer Algorithmus und Grafikkarte

Kardioide und Kreis überspringen

Punkte in der großen Kardioide und im Kreis links daneben liegen sicher in der Menge – eine einfache Formel erkennt sie ($q = (x - ¼)^2 + y^2$; in der Menge, wenn $q \cdot (q + x - ¼) \le y^2/4$ oder $(x + 1)^2 + y^2 \le 1/16$). Ohne Test

laufen genau diese rechenaufwendigsten Punkte bis zur maximalen Iterationszahl. Das Bild bleibt **exakt gleich** (in allen Rechenverfahren geprüft, auch direkt an den Rändern).

Ansicht	Rust, 1 Thread	Rust, 48 Threads	Rust SIMD, 1 Thread	Rust SIMD, 48 Threads
Gesamtansicht	6,38 → 0,73 s (8,7×)	0,24 → 0,11 s (2,3×)	1,67 → 0,29 s (5,7×)	0,068 → 0,032 s (2,1×)
Rand der Kardioide	27,7 → 2,0 s (13,6×)	0,80 → 0,21 s (3,7×)	6,9 → 0,64 s (10,8×)	0,21 → 0,053 s (3,9×)
Seepferdchental	unverändert	unverändert	unverändert	unverändert

> **Ein besserer Algorithmus schlägt mehr Kerne:** 1 Thread mit Test ist schneller als 8 Threads ohne. Mit vielen Threads ist der Gewinn kleiner, weil die übrige Arbeit ungleich verteilt und die Laufzeit nur noch Hundertstelsekunden ist.

Grafikkarte (RTX 3090)

Ein Direct3D-11-Compute-Shader (`native/gpu/mandel.hlsl`) rechnet jeden Bildpunkt in einem eigenen GPU-Thread, in Streifen zu 128 Zeilen (Windows bricht GPU-Aufträge nach ≈ 2 s ab). Die Farbe berechnet anschließend die CPU. Reine Rechenzeit ohne Bildübertragung:

Einstellung	Kardioide	Rust SIMD (48 Thr.)	GPU double	GPU float
1600 × 2560, 5 000 It.	nein	0,067 s	0,173 s (0,4×)	0,028 s (2,4×)
1600 × 2560, 5 000 It.	ja	0,031 s	0,101 s (0,3×)	0,027 s (1,2×)
1600 × 2560, 20 000 It.	nein	0,238 s	0,575 s (0,4×)	0,046 s (5,2×)
3840 × 6144, 5 000 It.	nein	0,288 s	0,846 s (0,3×)	0,109 s (2,6×)
3840 × 6144, 5 000 It.	ja	0,078 s	0,388 s (0,2×)	0,087 s (0,9×)

- **float:** bis 5× schneller als die beste CPU; die GPU selbst braucht nur 0,01–0,03 s, der Rest ist Zurückkopieren und Einfärben. Aber ungenauer: 0,3 % abweichende Pixel in der Gesamtansicht, 4–17 % in tiefen Zooms.
- **double:** langsamer als die CPU (0,2–0,4×) – die RTX 3090 rechnet double nur mit 1/64 der float-Geschwindigkeit (Rechenkarten wie A100/H100 deutlich besser). Auch hier weichen 0,01–0,5 % der Pixel ab; FMA ist nicht die Ursache (per Gegenprobe ausgeschlossen), der Grafiktreiber übersetzt den Shader auf eigene Weise.
- **Mit Kardioiden-Test** holt die CPU auf und ist beim 4K-Bild sogar schneller. GPU-Start ≈ 0,45 s. Die GPU-Last erscheint nicht in der CPU-Heatmap.

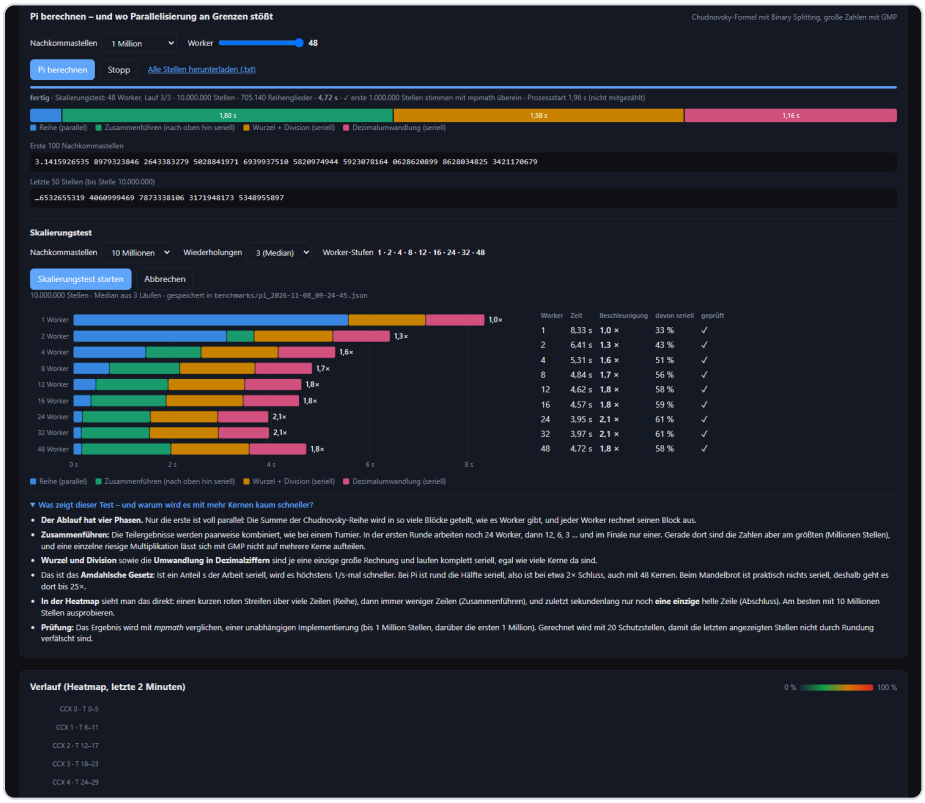
6. Pi und das Amdahlsche Gesetz

π wird mit der Chudnovsky-Formel und Binary Splitting berechnet (GMP über gmpy2). Nur die Summe der Reihe ist voll parallel; danach folgen Zusammenführen (nach oben hin immer weniger parallel), Wurzel und Division sowie die Umwandlung in Dezimalziffern – je eine einzige große Rechnung.

Worker	1	2	4	8	16	24	32	48
Gesamt (s)	8,33	6,41	5,31	4,84	4,57	3,95	3,97	4,72
Beschleunigung	1,0×	1,3×	1,6×	1,7×	1,8×	2,1×	2,1×	1,8×
Reihe, parallel (s)	5,56	3,10	1,47	0,73	0,36	0,18	0,16	0,17
Zusammenführen (s)	–	0,55	1,12	1,42	1,52	1,38	1,38	1,80
Wurzel + Division (s)	1,57	1,59	1,56	1,52	1,56	1,36	1,40	1,58
Dezimalumwandlung (s)	1,20	1,16	1,16	1,16	1,14	1,03	1,03	1,16

10 Millionen Nachkommastellen, Median aus 3. Alle Läufe stimmen mit mpmath überein (bis 1 Mio. Stellen vollständig geprüft).

> **Amdahlsches Gesetz:** Ist ein Anteil s der Arbeit seriell, wird es höchstens 1/s-mal schneller. Der parallele Teil schrumpft von 5,6 s auf 0,17 s (≈ 33×), aber ≈ 2,7 s für Wurzel, Division und Umwandlung bleiben auf einem Kern – dazu die größten Multiplikationen beim Zusammenführen. Ergebnis: höchstens ≈ 2,1×; mit 48 Workern wird es sogar wieder langsamer (mehr Zusammenführen, mehr Datentransport). In der Heatmap sieht man erst einen roten Streifen über alle Threads, dann immer weniger, und sekundenlang nur noch eine einzige helle Zeile.



Pi-Bereich: Phasenbalken, erste und letzte Stellen, Skalierungstest mit gestapelten Balken je Phase.

Gemessene Optimierungen: ein Block je Worker statt vier (4,6 statt 5,4 s), Zusammenführen auf die Worker verteilt statt im Hauptprozess (4,6 statt 5,8 s), im letzten Schritt entfällt eine der größten Multiplikationen. Ein C#- oder Rust-Gegenstück wurde bewusst nicht gebaut: Ohne GMP wären die Langzahl-Bibliotheken um ein Vielfaches langsamer – der Vergleich würde die Bibliothek messen, nicht die Parallelisierung.

7. Gefundene und behobene Fehler

Problem	Ursache	Lösung
Live-Ansicht fror nach „Last erzeugen – Start“ ein	Status während des Prozessstarts unvollständig → Fehler beendete die Mess-Schleife stillschweigend	Status vorab setzen, Mess-Schleife fängt Fehler ab, Start/Stopp/Status unter Sperre
„Auslagerungsdatei zu klein“, Worker stürzten ab	numpy/OpenBLAS legte in jedem Worker Puffer für 48 Threads an (≈ 800 MB je Worker, 37,6 GB bei 48); jeder Worker lud den ganzen Webserver	`OPENBLAS_NUM_THREADS=1`, winzige Startdatei, leichte Worker-Module: 62 MB je Worker (2,9 GB bei 48), Webserver 88 statt 834 MB
Server bei großen Pi-Rechnungen sekundenlang taub	GMP gibt das Python-GIL nicht frei	serielle Pi-Schritte in einem Worker-Prozess; gemessen max. 0,22 s Antwortzeit bei 50 Mio. Stellen
Verwaiste Worker-Prozesse nach Server-Ende	Windows beendet Pool-Kinder nicht mit	Worker überwachen ihren Elternprozess und beenden sich selbst
Erster Lauf nur mit 21 von 48 Workern	Pool war beim Start noch nicht vollständig	warten, bis alle Worker geantwortet haben
C#/Rust bei kurzen Laufzeiten ausgebremst	Kachelübergabe als base64-Text	binäre Übergabe (0,116 → 0,070 s bei SIMD)
Falsche Beschleunigung im Skalierungstest	Plan ohne Stufe „1 Worker“	1 Worker ist immer dabei
Direkt nach dem Entpacken einzelne Rechenverfahren und GPU-Erkennung mit „Zugriff verweigert“	Virenscanner sperrt frisch entpackte .exe-Dateien beim ersten Start einige Sekunden	Start wird bis zu 10 × im Sekundenabstand wiederholt; fehlt eine .exe im Paket Full, nennt die Seite den Virenscanner als wahrscheinliche Ursache; Hinweis in Kapitel 8 und INSTALLATION.de.txt
Alter Server an 0.0.0.0 (im Netzwerk erreichbar)	erster Prototyp	Adresse fest 127.0.0.1
Veraltete Seitendateien im Browser	Browser-Cache	Cache-Header, automatisches Neuladen bei Serverneustart oder Dateiänderung
Paket startete fremde Builds bzw. scheiterte ohne Werkzeuge	Zeitstempel nach dem Kopieren	mitgelieferte Programme verwenden, wenn Werkzeuge fehlen; Toleranz 1 Minute

8. Weitergabe: Pakete Full und Lite

Es gibt zwei Weitergabe-Pakete: „Full“ (Komplett, alles fertig übersetzt) und „Lite“ (Klein, nur Quelltexte).

	Paket Full	Paket Lite
Bauen mit	`export_full.bat`	`export_lite.bat`
Ergebnis	`export\CoreMonitor_Full_<Datum>.zip`	`export\CoreMonitor_Lite_<Datum>.zip`
Größe	ZIP 104 MB, entpackt 281 MB	ZIP 0,1 MB, entpackt 0,3 MB
Inhalt	Python 3.11.9 (eingebettet) mit allen Paketen; alle 13 Rechenverfahren fertig übersetzt (C/C++/Assembler/GPU mit statischer Laufzeitbibliothek, vorübersetzte Shader, Rust statisch, C# als eigenständige .exe); Quelltexte; mpmath-Referenz	nur Quelltexte und Anleitung
Ziel-PC braucht	nur Windows 10/11 (64 Bit) – kein Python, kein Internet, keine Admin-Rechte	Python 3.11+ und einmal Internet; optional .NET SDK, Rust, Visual Studio C++
Getestet	fremder Ordner, leerer PATH: alle 13 Rechenverfahren, Pi, Last, Skalierungstest; simulierter kleiner PC (8 Threads, kein AVX2, keine GPU)	mit allen Werkzeugen (alles übersetzt, 13 Rechenverfahren, Start 27 s) und nur mit Python (2 Rechenverfahren, 11 ausgegraut mit Hinweis)

Eigene Messdaten, Einstellungen, Protokolle und Zwischendateien kommen in keines der Pakete. Beide enthalten `README.md` (Englisch) und `README.de.md`, `INSTALLATION.txt` (Englisch) und `INSTALLATION.de.txt` sowie beide Projektberichte als PDF: `Project_Report_Core_Monitor.pdf` (Englisch) und `Projektbericht_Parallelisierung.pdf` (Deutsch).

> **Hinweis zum Virenscanner: unbegründete Warnungen möglich.** Virenscanner (z. B. Microsoft Defender, Trend Micro) und Windows SmartScreen können bei diesem Programm Alarm schlagen oder Teile blockieren – ein Fehlalarm. Auslöser sind Verhaltensmuster, die auch Schadprogramme zeigen: selbst übersetzte `.exe`-Dateien ohne Signatur und Bekanntheit, frisch entpackte Dateien, bis zu 48 gleichzeitig gestartete Prozesse und 100 % Auslastung aller Kerne (ähnlich wie ein Krypto-Miner). **Anzeichen:** „Zugriff verweigert“ oder langsamer erster Start; `.exe` in Quarantäne (Rechenverfahren ausgegraut mit Hinweis); SmartScreen-Warnung. **Was tun:** kurz warten bzw. `start.bat` neu starten (der Monitor wiederholt gesperrte Starts bis zu 10 s); ZIP vor dem Entpacken unter Eigenschaften „Zulassen“; Datei aus der Quarantäne wiederherstellen; den Ordner als Ausnahme eintragen – nur bei vertrauenswürdiger Herkunft (selbst gebaut oder direkt vom Ersteller); zur Kontrolle `.exe`-Dateien bei VirusTotal prüfen oder das Paket Lite selbst übersetzen. Den Virenscanner nicht abschalten. numba, numpy und Pi laufen auch ohne die `.exe`-Dateien.

Installation für das Paket Lite (Schritt für Schritt)

Schritt	Wofür	Wie	Prüfen
1. Python 3.11+ (Pflicht)	alles	python.org → „Windows installer (64-bit)“, „Add python.exe to PATH“ anhaken; oder `winget install Python.Python.3.11`	`python --version`
2. Python-Pakete	alles	macht `start.bat` automatisch (fastapi, uvicorn, psutil, numpy, gmpy2, mpmath, numba)	Seite startet
3. .NET SDK 10 (optional)	C#, C# SIMD	dotnet.microsoft.com oder `winget install Microsoft.DotNet.SDK.10`	`dotnet --version`
4. Rust (optional)	Rust, Rust SIMD	rustup.rs oder `winget install Rustlang.Rustup` (braucht den Linker aus Schritt 5)	`cargo --version`
5. Visual Studio C++ (optional)	C, C++, Assembler, GPU	Build Tools oder Community, Arbeitsauslastung „Desktopentwicklung mit C++“ (cl, ml64, Windows SDK mit fxc)	`native\build.bat`
6. Grafikkartentreiber (optional)	GPU	aktueller Treiber von NVIDIA, AMD oder Intel (Direct3D 11)	„System“-Box

Die Seite erkennt CPU und Grafikkarte beim Start; SIMD und Assembler brauchen AVX2. Was fehlt, erscheint ausgegraut mit Grund. Das Paket Full braucht nichts davon: entpacken, `start.bat` doppelklicken.

Lizenz, Sicherheit, Entstehung

- **Lizenz:** MIT (Datei `LICENSE`). Das Paket „Full“ enthält Software Dritter unter deren eigenen Lizenzen (u. a. Python, numpy, numba, gmpy2 mit GMP/MPFR/MPC unter LGPL, .NET-Laufzeit, Microsoft-C/C++-Laufzeit) – Übersicht in `THIRD_PARTY_NOTICES.md`, vollständige Lizenztexte im Ordner `licenses\` des Pakets. Zu jedem Paket gibt es eine SHA-256-Prüfsumme (`.sha256`).
- **Schutz vor Anfragen von außen:** Neben der Bindung an 127.0.0.1 prüft der Server bei jeder Anfrage Absender, Host und Herkunft (Origin). Fremde Anfragen – etwa per DNS-Rebinding oder von einer fremden Webseite im Browser – werden mit 403 abgewiesen, in `logs/security.log` protokolliert und gemeldet (Konsole und roter Hinweis in der Kopfzeile). Der Rechnername wird nicht mehr übertragen.
- **Automatische Tests:** GitHub Actions übersetzt bei jedem Push alle Rechenverfahren und führt den Selbsttest aus (bitgenaue Bilder, Pi, Sprachumschaltung, Schutzfunktionen, Datenschutz-Prüfung).
- **Entstehung:** Projekt, Code, Dokumentation und Berichte wurden mit Claude (Anthropic) über Claude Code umgesetzt, gesteuert vom Projektinhaber.

9. Grenzen und Ideen

- **Windows und Linux** (Debian / Ubuntu, Kapitel 10); macOS wird nicht unterstützt.
- **Über 64 Threads** (mehrere Prozessorgruppen) wird nur die Platzierung „Windows“ angeboten.
- **GPU-Bilder** weichen leicht ab (double 0,01–0,5 %, float bis 17 % in Zooms); double ist auf Spielegrafikkarten langsam.
- **Weitere Ideen:** mehrere Pixelgruppen verschränkt rechnen (Latenz verdecken), kleinere Kacheln nur am Ende, Periodizitätsprüfung, CUDA/OpenCL statt Direct3D, Mandelbrot in Multi-Precision für tiefe Zooms, Ausführung auf einem Rechner mit AVX-512 oder einer Rechen-GPU zum Vergleich.

Messwerte stammen vom Referenzrechner; auf anderen Rechnern liefert der eingebaute Skalierungstest die eigenen Werte.

10. Linux-Fassung (Debian / Ubuntu)

Seit November 2026 läuft derselbe Ordner auch unter Linux – dieselbe Seite, dieselben 13 Rechenverfahren, dieselben Bilder. Gestartet wird mit `./start.sh`: Das Skript legt eine eigene Python-Umgebung an (`.venv`, das System-Python bleibt unverändert), installiert die Pakete und übersetzt, was die vorhandenen Werkzeuge erlauben (`native/build.sh`, cargo, dotnet). Fehlende Werkzeuge grauen nur die betroffenen Rechenverfahren aus. Unterstützt: Debian 12 und Ubuntu 22.04 oder neuer (x86-64), Python 3.10+.

	Windows	Linux
Start	`start.bat`	`./start.sh` (Pakete in `.venv`)
C, C++	MSVC (Visual Studio)	GCC (`build-essential`)
Threads	Win32-Threads	POSIX-Threads
Assembler	MASM, `kernel.asm`	NASM, `kernel_nasm.asm` – System-V-Aufrufkonvention, dieselben Befehle
GPU	Direct3D 11 Compute Shader	OpenCL (`mandel.cl`)
Topologie (Kerne, Cache-Gruppen)	`GetLogicalProcessorInformationEx`	`/sys/devices/system/cpu`
Platzierung (Affinität)	`SetThreadAffinityMask`	`sched_setaffinity`
Priorität der Rechnungen	„niedriger als normal“	nice
Weitergabe-Pakete	ZIP (`export_*.bat`)	tar.gz (`export_*.sh`)

Ergebnisse

- **Ubuntu 24.04** (Test-Rechner in der Cloud): alle 11 CPU-Rechenverfahren bitgenau gleich zu numba, auch mit Python 3.10. OpenCL über POCL (OpenCL-Treiber für die CPU): „GPU double“ bitgenau, bei „GPU float“ weichen ≈ 0,6 % der Bildpunkte ab (Testbild 640 × 400).
- **Referenzrechner in WSL 1** (Ubuntu 22.04, Python 3.10): 9 Rechenverfahren (numba, numpy, C, C SIMD, C++, C++ SIMD, Assembler, Rust, Rust SIMD) bitgenau gleich zu numba; C# entfällt, weil dort kein .NET installiert ist. Die GPU ist ausgegraut mit Begründung – WSL hat keinen OpenCL-Zugriff auf die Grafikkarte. Selbsttest (Sprache, Schutz vor Anfragen von außen, Skalierung, Pi, Datenschutz) vollständig bestanden.
- **Windows** verhält sich nach dem Umbau unverändert: alle 13 Rechenverfahren und der Selbsttest bestanden.

Erkenntnisse und Stolpersteine

- **Andere Thread-Nummerierung:** Linux nummeriert die SMT-Geschwister vieler Systeme anders als Windows (AMD: Kern 0 = Threads 0 und 24). Kernansicht, Heatmap und Platzierungen folgen deshalb der echten Topologie aus `/sys`. In WSL 1 sind die Paare wie unter Windows 0/1, 2/3 …
- **WSL 1 meldet keine L3-Cache-Gruppen und keinen Takt:** Die Seite bildet dann Gruppen zu 4 Kernen und zeigt beim Takt „–“. Ein Fehler dabei (psutil brach beim Lesen des Takts ab und verhinderte den Serverstart) wurde im WSL-Test gefunden und behoben.

- **Worker-Prozesse starten mit** `spawn` (wie unter Windows), nicht mit `fork` – sicherer neben den Threads des Webservers.
- **Bitgenauigkeit:** GCC braucht `-ffp-contract=off`, sonst fasst es Multiplikation und Addition zu FMA zusammen und einzelne Bildpunkte ändern sich. Der OpenCL-Kernel schaltet dasselbe mit `#pragma OPENCL FP_CONTRACT OFF` ab.

Pakete und automatische Tests

- **Linux Lite** ≈ 1 MB (nur Quelltexte). **Linux Full** ≈ 150 MB als tar.gz, entpackt ≈ 470 MB: Python 3.11 aus python-build-standalone, C# als eigenständiges Programm, C/C++ mit statisch gebundener C++-Laufzeit. Gebaut auf Ubuntu 22.04 (glibc 2.35), damit es auch auf älteren Systemen läuft; das Ziel braucht weder Python noch Internet noch sudo.
- **GitHub Actions** hat einen Ubuntu-Lauf mit POCL (alle 13 Rechenverfahren); der Release-Workflow baut das Paket Linux Full.

> **Noch offen:** Leistungsmessungen unter echtem Linux fehlen noch. WSL 1 ist dafür nicht aussagekräftig (Übersetzungsschicht statt Linux-Kernel); alle Messwerte in diesem Bericht stammen von Windows.

11. Begriffe kurz erklärt

Alphabetisch, mit Link zu einer weiterführenden Quelle.

0.0.0.0	Server-Einstellung „auf allen Netzwerkkarten erreichbar“. Der Monitor nutzt das bewusst nicht, sondern nur 127.0.0.1.
/sys und /proc	Virtuelle Ordner, über die Linux Auskunft über die Hardware gibt; daraus liest der Monitor Kerne, Threads und L3-Gruppen. https://www.kernel.org/doc/html/latest/filesystems/sysfs.html
127.0.0.1 (localhost)	Adresse des eigenen PCs. Der Server lauscht nur dort und ist deshalb von anderen Rechnern aus nicht erreichbar. https://de.wikipedia.org/wiki/Localhost
Amdahlsches Gesetz	Der nicht parallelisierbare Anteil eines Programms begrenzt die mögliche Beschleunigung, egal wie viele Kerne man einsetzt (siehe Pi). https://de.wikipedia.org/wiki/Amdahlsches_Gesetz
API	Schnittstelle, über die die Seite (oder ein Skript) den Server steuert, z. B. /api/mandel/start. https://de.wikipedia.org/wiki/Programmierschnittstelle
apt	Paketverwaltung von Debian und Ubuntu; installiert z. B. Compiler und Python-Werkzeuge (sudo apt install …). https://wiki.debian.org/de/Apt
Assembler (MASM)	Prozessorbefehle in lesbarer Form, von Hand geschrieben. MASM ist der Microsoft Macro Assembler. https://learn.microsoft.com/de-de/cpp/assembler/masm/microsoft-macro-assembler-reference
Aufrufkonvention	Regel, in welchen Registern eine Funktion ihre Argumente bekommt. Windows und Linux unterscheiden sich, daher gibt es den Assembler-Kern zweimal. https://en.wikipedia.org/wiki/X86_calling_conventions
„Aufwendige Kacheln zuerst“	Reihenfolge, bei der eine grobe Vorab-Schätzung die rechenaufwendigsten Kacheln an den Anfang stellt, damit am Ende keine lange Kachel allein übrig bleibt.
AVX-512	Nachfolger von AVX2 mit 512-Bit-Registern (8 double gleichzeitig). Der Threadripper 3960X hat es nicht; neuere Prozessoren teils schon. https://de.wikipedia.org/wiki/AVX-512
AVX2	SIMD-Befehlssatz aktueller x86-Prozessoren; 256-Bit-Register fassen 4 Kommazahlen (double), also 4 Bildpunkte je Befehl. https://de.wikipedia.org/wiki/Advanced_Vector_Extensions
base64	Verfahren, Binärdaten (hier Bildkacheln) als reinen Text zu verschicken; etwa ein Drittel größer als das Original. https://de.wikipedia.org/wiki/Base64
Binary Splitting	Rechentrick, der die Reihenglieder paarweise zu großen Brüchen zusammenfasst; die Teilstücke lassen sich parallel berechnen, die letzte Division nicht. https://en.wikipedia.org/wiki/Binary_splitting

bitgenau	Jedes Byte des Ergebnisses ist identisch – hier: 0 Pixel Unterschied zwischen den Rechenverfahren.
C / C++	Klassische, maschinennahe Sprachen. Unter Windows mit dem Microsoft-Compiler (MSVC) aus Visual Studio übersetzt, unter Linux mit GCC. https://learn.microsoft.com/de-de/cpp/
C# / .NET	Programmiersprache und Laufzeitumgebung von Microsoft; echte Threads, Übersetzung in Maschinencode zur Laufzeit. https://learn.microsoft.com/de-de/dotnet/csharp/
Cache (L1/L2/L3)	Schneller Zwischenspeicher im Prozessor. L1/L2 gehören je einem Kern, L3 teilen sich die Kerne eines CCX. https://de.wikipedia.org/wiki/Cache
CCX (Core Complex)	Bei AMD-Zen-Prozessoren eine Gruppe von Kernen mit gemeinsamem L3-Cache; beim 3960X 8 CCX zu je 3 Kernen. Im Monitor „L3-Gruppe“. https://en.wikipedia.org/wiki/Zen_2
Chudnovsky-Formel	Sehr schnell konvergierende Reihe für Pi: etwa 14 neue Stellen je Glied. https://en.wikipedia.org/wiki/Chudnovsky_algorithm
Compiler-Schalter /O2, /arch:AVX2, /fp:precise, /MT	Optimieren, AVX2-Befehle erlauben, Rechnung exakt wie geschrieben (kein FMA), Laufzeitbibliothek fest einbinden (keine Zusatz-DLLs nötig). https://learn.microsoft.com/de-de/cpp/build/reference/compiler-options-listed-by-category
CPU-Affinität (Platzierung)	Festlegung, auf welchen Threads ein Prozess laufen darf. Damit testet der Monitor „1 Thread je Kern zuerst“ gegen „kompakt“. https://en.wikipedia.org/wiki/Processor_affinity
Digitale Signatur (Code-Signatur)	Elektronische Unterschrift eines Herstellers an einer .exe; signierte, verbreitete Programme lösen seltener Warnungen aus. Die Programme des Kern-Monitors sind nicht signiert. https://en.wikipedia.org/wiki/Code_signing
Direct3D 11 Compute-Shader / HLSL	Programm, das auf der Grafikkarte läuft; geschrieben in HLSL, gestartet über Direct3D 11 (funktioniert mit jeder aktuellen Grafikkarte unter Windows). https://learn.microsoft.com/de-de/windows/win32/direct3d11/direct3d-11-advanced-stages-compute-shader
DNS-Rebinding	Angriff, bei dem eine fremde Webseite ihren Namen auf 127.0.0.1 umleitet, um aus dem Browser heraus lokale Dienste zu steuern. Der Monitor prüft deshalb bei jeder Anfrage Host und Herkunft und weist Fremdes ab. https://en.wikipedia.org/wiki/DNS_rebinding
double / float	Kommazahlen mit doppelter (≈ 16 Stellen) bzw. einfacher Genauigkeit (≈ 7 Stellen) nach IEEE 754. Spiele-Grafikkarten sind mit float viel schneller. https://de.wikipedia.org/wiki/IEEE_754
DXGI	Windows-Schnittstelle, über die der Monitor die vorhandenen Grafikkarten und ihren Speicher erkennt. https://learn.microsoft.com/de-de/windows/win32/direct3ddxgi/dx-graphics-dxgi
Embedded Python	Offizielles, schlankes Python zum Mitgeben in einem Ordner, ohne Installation. Steckt im Windows-Paket „Komplett“ (unter Linux: python-build-standalone). https://docs.python.org/3/using/windows.html#the-embeddable-package
FastAPI / uvicorn	Python-Bausteine für den Webserver: FastAPI beschreibt die Schnittstellen, uvicorn liefert sie aus. https://fastapi.tiangolo.com/

Fehlalarm (False Positive)	Ein Virenscanner hält ein harmloses Programm für gefährlich, weil es sich ähnlich verhält wie Schadsoftware (z. B. unsignierte .exe, volle Auslastung aller Kerne). Siehe „Hinweis zum Virenscanner“. https://de.wikipedia.org/wiki/Falsch_positiv
FMA (Fused Multiply-Add)	Multiplikation und Addition in einem Schritt mit nur einer Rundung. Schneller, verändert aber die letzte Stelle – und damit einzelne Bildpunkte. https://en.wikipedia.org/wiki/Multiply%E2%80%93accumulate_operation
GCC	GNU Compiler Collection: freier Compiler für C und C++, unter Linux der Standard. Schalter -ffp-contract=off verhindert FMA, damit die Bilder bitgenau bleiben. https://gcc.gnu.org/
GIL (Global Interpreter Lock)	Sperre im Standard-Python: Nur ein Thread führt gleichzeitig Python-Code aus. Deshalb nutzt der Monitor für Python Prozesse statt Threads. https://docs.python.org/3/glossary.html#term-global-interpreter-lock
glibc	Die C-Standardbibliothek von Linux. Fertige Programme laufen nur mit gleicher oder neuerer glibc, deshalb wird auf dem ältesten unterstützten System gebaut. https://www.gnu.org/software/libc/
gmpy2 / GMP	Bibliothek für sehr große Zahlen mit beliebig vielen Stellen; Grundlage der Pi-Berechnung. https://gmpy2.readthedocs.io/
GPU (Grafikkarte)	Prozessor mit Tausenden einfacher Recheneinheiten; ideal, wenn sehr viele gleichartige Rechnungen anfallen (z. B. je Bildpunkt eine). https://de.wikipedia.org/wiki/Grafikprozessor
Heatmap	Farbige Kachelansicht: je Thread ein Feld, Farbe = aktuelle Auslastung. https://de.wikipedia.org/wiki/Heatmap
Intrinsics	Funktionen in C/C++/Rust, die direkt einem Prozessorbefehl (z. B. AVX2) entsprechen; SIMD ohne Assembler. https://learn.microsoft.com/de-de/cpp/intrinsics/compiler-intrinsics
Iterationen (max_iter)	Wie oft $z \to z^2 + c$ höchstens wiederholt wird. Mehr Iterationen = feinere Ränder, aber mehr Rechenzeit (besonders im schwarzen Inneren).
JIT (Just-in-time-Übersetzung)	Code wird erst während des Programmlaufs in Maschinencode übersetzt; der erste Aufruf dauert daher länger. https://de.wikipedia.org/wiki/Just-in-time-Kompilierung
JSON / CSV	Textformate für Messdaten: JSON für strukturierte Daten (eine Datei je Messung), CSV als Tabelle, die z. B. Excel öffnen kann. https://de.wikipedia.org/wiki/JavaScript_Object_Notation
Kachel	Rechteckiger Bildausschnitt (Standard 128 × 128 Pixel), die kleinste Arbeitseinheit eines Workers.
Kardioide und Periode-2-Kreis	Die herzförmige Hauptfläche und der große Kreis links davon gehören sicher zur Mandelbrot-Menge. Eine kurze Formel erkennt diese Punkte, ohne sie bis zur Höchstzahl durchzurechnen. https://de.wikipedia.org/wiki/Kardioide
Kern (Prozessorkern)	Eigenständiges Rechenwerk im Prozessor. Der Threadripper 3960X hat 24 Kerne. https://de.wikipedia.org/wiki/Mehrkernprozessor
Lastverteilung (dynamisch)	Wer mit seiner Kachel fertig ist, holt sich die nächste. So bleiben keine Kerne untätig, obwohl Kacheln unterschiedlich lange dauern. https://de.wikipedia.org/wiki/Lastverteilung_(Informatik)

Linux (Debian / Ubuntu)	Freies Betriebssystem; der Kern-Monitor läuft auch unter Debian 12 und Ubuntu 22.04 oder neuer (start.sh). https://de.wikipedia.org/wiki/Linux
Mandelbrot-Menge	Bekanntes Fraktal: Für jeden Bildpunkt wird $z \to z^2 + c$ wiederholt, bis der Wert „entkommt" oder die Höchstzahl erreicht ist. Jeder Punkt ist unabhängig – ideal für Parallelisierung. https://de.wikipedia.org/wiki/Mandelbrot-Menge
Median	Mittlerer Wert einer sortierten Messreihe; unempfindlicher gegen Ausreißer als der Durchschnitt. Wird bei mehreren Läufen gewertet. https://de.wikipedia.org/wiki/Median
mpmath	Python-Bibliothek für Rechnen mit beliebiger Genauigkeit; dient als unabhängige Gegenprobe für die Pi-Stellen. https://mpmath.org/
NASM	Netwide Assembler: freier x64-Assembler; unter Linux übersetzt er die handgeschriebene Rechenschleife (asm/kernel_nasm.asm). https://www.nasm.us/
nice (Priorität)	Linux-Wert für die Priorität eines Prozesses; die Rechen-Worker laufen mit nice 5 bzw. 10, damit Browser und Server flüssig bleiben. https://de.wikipedia.org/wiki/Nice_(Unix)
NUMA	Non-Uniform Memory Access: Bei großen Systemen ist Speicher je nach Kern unterschiedlich schnell erreichbar. Der 3960X hat einen NUMA-Knoten. https://de.wikipedia.org/wiki/Non-Uniform_Memory_Access
numba	Übersetzt ausgewählte Python-Funktionen beim ersten Aufruf in Maschinencode (JIT) und macht sie so etwa so schnell wie C. https://numba.pydata.org/
numpy	Python-Bibliothek für Rechnen mit ganzen Zahlenfeldern (Arrays). Schnell bei einfachen Feldoperationen, aber nicht ideal für Mandelbrot. https://numpy.org/
OpenBLAS	Mathe-Bibliothek in numpy, die selbst Threads startet. Im Monitor auf 1 Thread je Prozess begrenzt, sonst braucht jeder Worker ~800 MB. https://www.openblas.net/
OpenCL	Offener Standard, um Programme auf Grafikkarten (und Prozessoren) laufen zu lassen; unter Linux der Weg zur GPU statt Direct3D. https://www.khronos.org/opencl/
pip	Paketverwaltung von Python; installiert Erweiterungen wie numba oder FastAPI. https://pip.pypa.io/
POCL	Portable Computing Language: OpenCL-Treiber, der auf dem Prozessor rechnet. Der automatische Test nutzt ihn als Ersatz-Grafikkarte. https://portablecl.org/
POSIX-Threads (pthreads)	Standard-Schnittstelle für Threads unter Linux; das Gegenstück zu CreateThread unter Windows. https://de.wikipedia.org/wiki/POSIX_Threads
ProcessPoolExecutor	Python-Baustein, der einen Vorrat an Worker-Prozessen verwaltet und Aufgaben an sie verteilt. https://docs.python.org/3/library/concurrent.futures.html

Prozess	Laufendes Programm mit eigenem Speicher. Mehrere Prozesse laufen unabhängig, tauschen Daten aber nur über Umwege aus. https://de.wikipedia.org/wiki/Prozess_(Informatik)
Prozessorgruppe	Windows fasst höchstens 64 Threads zu einer Gruppe zusammen. Prozessoren mit mehr Threads haben mehrere Gruppen; ein Prozess läuft normalerweise nur in einer. https://learn.microsoft.com/de-de/windows/win32/procthread/processor-groups
psutil	Python-Bibliothek, die Auslastung, Speicher und Prozesse abfragt; liefert die Werte für die Kern-Anzeige. https://psutil.readthedocs.io/
Python	Verbreitete Skriptsprache; hier für Webserver, Steuerung und zwei Rechenverfahren. https://www.python.org/
python-build-standalone	Fertig übersetztes Python, das man ohne Installation in einen Ordner legen kann; steckt im Linux-Paket „Komplett“. https://github.com/astral-sh/python-build-standalone
Quarantäne	Abgeschotteter Bereich des Virenscanners für verdächtige Dateien; von dort lassen sie sich wiederherstellen. https://de.wikipedia.org/wiki/Antivirenprogramm
Rust	Moderne Systemsprache mit C-ähnlicher Geschwindigkeit; der Übersetzer verhindert typische Fehler beim Arbeiten mit Threads. https://www.rust-lang.org/
rustup / cargo	rustup installiert und aktualisiert Rust; cargo übersetzt Rust-Programme und lädt benötigte Bibliotheken. https://rustup.rs/
Scheduler	Teil des Betriebssystems, der entscheidet, welcher Thread wann auf welchem Kern läuft (Platzierung „Windows entscheidet“ bzw. „Linux entscheidet“). https://de.wikipedia.org/wiki/Prozess-Scheduler
Seepferdchental	Bekannter Ausschnitt der Mandelbrot-Menge zwischen Hauptfläche und Periode-2-Kreis; sehr fein verästelt und daher rechenaufwendig. https://de.wikipedia.org/wiki/Mandelbrot-Menge
Self-contained Single-File	.NET-Programm, das die Laufzeitumgebung in einer einzigen Datei mitbringt (unter Windows eine .exe); läuft ohne installiertes .NET. https://learn.microsoft.com/de-de/dotnet/core/deploying/single-file/overview
SIMD	Single Instruction, Multiple Data: Ein Befehl rechnet mehrere Zahlen gleichzeitig. https://de.wikipedia.org/wiki/SIMD
Skalierung, Beschleunigung, Effizienz	Beschleunigung = Zeit mit 1 Worker ÷ Zeit mit n Workern. Effizienz = Beschleunigung ÷ n (100 % = perfekt). https://de.wikipedia.org/wiki/Speedup
SmartScreen	Windows-Schutz, der vor unbekannten, nicht signierten Programmen warnt („Der Computer wurde durch Windows geschützt“). https://learn.microsoft.com/de-de/windows/security/operating-system-security/virus-and-threat-protection/microsoft-defender-smartscreen/
SMT (Simultaneous Multithreading, bei Intel „Hyper-Threading“)	Ein Kern bearbeitet zwei Befehlsströme gleichzeitig und füllt damit Lücken in seinen Rechenwerken. Bringt meist 0–30 %, nicht 100 %. https://de.wikipedia.org/wiki/Simultaneous_Multithreading

Thread (Hardware-Thread, logischer Prozessor)	Was das Betriebssystem (Windows, Linux) als „CPU“ zählt. Mit SMT bietet jeder Kern 2 Threads, also 48 beim 3960X. https://de.wikipedia.org/wiki/Thread_(Informatik)
Thread (Software)	Ausführungsstrang innerhalb eines Prozesses; alle Threads teilen sich den Speicher und starten viel schneller als Prozesse. https://de.wikipedia.org/wiki/Thread_(Informatik)
vcomp140.dll / OpenMP	OpenMP ist ein Standard, mit dem C/C++-Programme Schleifen auf Threads verteilen; vcomp140.dll ist Microsofts Laufzeitbibliothek dafür und liegt dem Paket bei. https://learn.microsoft.com/de-de/cpp/parallel/openmp/openmp-in-visual-cpp
Vector256	.NET-Datentyp für ein 256-Bit-Register (4 double); damit nutzt C# AVX2. https://learn.microsoft.com/de-de/dotnet/api/system.runtime.intrinsics.vector256
venv (Python-Umgebung)	Eigener Ordner mit Python-Paketen nur für ein Programm (.venv); start.sh installiert dort, das System-Python bleibt unverändert. https://docs.python.org/3/library/venv.html
VirusTotal	Kostenloser Dienst, der eine Datei mit rund 70 Virenscannern gleichzeitig prüft. https://www.virustotal.com/
Visual Studio Build Tools	Microsofts Compiler für C, C++ und Assembler ohne die volle Entwicklungsumgebung; kostenlos. https://visualstudio.microsoft.com/de/downloads/
WebSocket	Dauerhafte Verbindung zwischen Browser und Server; darüber kommen die Auslastungswerte jede Sekunde ohne Neuladen. https://de.wikipedia.org/wiki/WebSocket
winget	Paketverwaltung von Windows; installiert Programme wie Python, .NET oder Rust per Befehlszeile. https://learn.microsoft.com/de-de/windows/package-manager/winget/
Worker	Ein Prozess oder Thread, der Kacheln abarbeitet. Die Worker-Zahl stellt man im Monitor ein (1–48).
WSL (Windows-Subsystem für Linux)	Linux direkt unter Windows; der Kern-Monitor läuft darin, aber ohne Zugriff auf die Grafikkarte über OpenCL. https://learn.microsoft.com/de-de/windows/wsl/
x64	64-Bit-Befehlssatz der heutigen Intel- und AMD-Prozessoren (auch x86-64 oder AMD64). https://de.wikipedia.org/wiki/X64